

HW Week1: Day 1 ~ 3 Labs Report

김종원

Lab 과제들 기술에 앞서 Verilog 문법을 정리해 보았다.

Verilog 기본 문법 정리

1. Verilog란?

Verilog HDL(Hardware Description Language)은 디지털 하드웨어 회로를 설계하고 동작을 기술하기 위한 하드웨어 기술 언어이다. 일반적인 소프트웨어 언어가 명령어를 순차적으로 실행하는 것과 달리, Verilog는 여러 회로 블록이 동시에 동작하는 하드웨어의 병렬성을 기술하는 데 사용된다. 따라서 CPU, 메모리, ALU, 레지스터 파일과 같은 디지털 회로를 설계하고 시뮬레이션 하는 데 사용된다.

2. Module

Verilog에서 회로의 기본 단위는 module이다. 하나의 module은 독립적인 하드웨어 블록을 의미하며, 입력과 출력 포트를 가진다.

```
module adder(  
    input [31:0] a,  
    input [31:0] b,  
    output [31:0] sum  
);  
  
endmodule
```

위 예제는 두 입력값을 받아 합을 출력하는 모듈이다.

구성 요소는 다음과 같다.

- module : 모듈 선언 시작
 - input : 입력 포트
 - output : 출력 포트
 - endmodule : 모듈 종료
-

3. Data Type

Verilog에서는 주로 wire와 reg를 사용한다.

(1) Wire

wire는 물리적인 배선을 의미하며, continuous assignment(assign)나 다른 모듈의 출력에 의해 값이 결정된다.

```
wire [31:0] result;
```

- 항상 외부에서 값이 결정됨
- assign과 함께 자주 사용됨

예시로는

```
assign result = a + b;
```

(2) Reg

reg는 값을 저장할 수 있는 변수이다. 주로 always 블록 내부에서 사용된다.

```
reg [31:0] data;
```

다음과 같은 특징을 가진다.

- 이전 값을 유지 가능
- sequential logic 구현 가능

예시로는

```
always @(posedge clk)
    data <= input_data;
```

4. Bit Width

Verilog에서는 비트 폭을 명시할 수 있다.

[MSB:LSB]

Ex

```
wire [31:0] a;
```

이는 32비트 신호를 의미한다. (0~31)

특정 비트 접근:

a[0]

슬라이싱:

a[31:16]

5. Number Representation

Verilog 숫자 표기:

<비트수>'<진법><값>

예시:

4'b1010

8'hFF

32'd100

각 알파벳의 의미는 다음과 같다:

- b : binary
- h : hexadecimal
- d : decimal

예:

32'hFFFFFFFF

32비트 16진수 값.

16진수의 각 자리 F는 decimal 15이며, binary로는 1111에 해당한다.

6. Assign Statement

assign은 combinational logic 표현에 사용된다.

assign y = a & b;

회로로는

a ----)

| AND (---- y

b ----)

즉 입력이 바뀌면 출력도 즉시 변한다.

ALU 예시:

```
assign sub_res_w = alu_a_i - alu_b_i;
```

이는 두 입력 차를 항상 계산한다.

연결이 지속 되어있는 상태기에 입력이 바뀌면 그 즉시 출력도 변화하게 된다.

7. Always Block

always 블록은 특정 조건 변화 시 실행된다.

기본 형태:

```
always @(sensitivity list)
```

```
begin
```

```
...
```

```
end
```

두 종류가 중요하다.

(1) Combinational Logic

입력 변화 시 즉시 반응.

```
always @(a or b)
```

```
begin
```

```
    out = a & b;
```

```
end
```

또는

always @(*)

사용함

메모리 없음

- 입력만으로 출력 결정

(2) Sequential Logic

Clock edge에서 동작.

always @(posedge clk)

begin

...

end

주 특징:

- flip-flop
- register
- state machine

예시:

always @(posedge clk)

count <= count + 1;

posedge: positive edge = 신호가 0 → 1로 바뀌는 순간

8. Blocking / Non-blocking Assignment

두 assignment 방식이 있다.

Blocking (=)

순차 실행.

a = b;

c = a;

앞 문장이 끝나야 다음 문장 실행.

Non-blocking (<=)

동시에 업데이트.

```
a <= b;
```

```
c <= a;
```

Sequential circuit에서는 일반적으로 non-blocking 사용.

규칙:

- combinational → =
 - sequential → <=
-

9. Conditional Statement

if문 사용 가능.

```
if (enable)
```

```
    out = a;
```

```
else
```

```
    out = b;
```

예시:

```
if (dwr_i)
```

```
    mem[address] <= data;
```

이러한 점에서 C언어와 유사하다고 하는 것 같다.

10. Case Statement

여러 조건 분기.

```
case(opcode)
```

```
2'b00: result = a+b;
```

```
2'b01: result = a-b;
```

```
endcase
```

Opcode 값이 2'b00이면 첫번째 줄... 이런 식으로 C의 switch-case와 거의 비슷하다.

ALU opcode decoding에 자주 사용한다고 한다.

예시:

```
case (alu_op_i)
```

```
    ALU_ADD:
```

```
    ALU_SUB:
```

```
endcase
```

11. Concatenation

여러 비트를 붙이는 연산.

{a, b}

예시:

{a, b}에서

a = 4'b1010

b = 4'b1100 이면,

이어 붙여서 10101100

즉 8'b10101100이 됨.

또는 shift 구현:

{alu_a_i[30:0], 1'b0}

이는 left shift 1과 동일하다.

12. Replication

같은 값을 반복.

```
{4{8'b0}}
```

예:

```
{4{dwdata_i[7:0]}}
```

Byte write 시 데이터를 4번 복사한다.

13. Memory Array

배열 형태 메모리 선언.

```
reg [31:0] mem [0:1023];
```

의미:

- 각 원소는 32비트
- 총 1024개 word 저장

... 32비트 데이터를 저장할 수 있는 공간 1024개

메모리 읽기:

mem[address] → 주소 값 리턴

메모리 쓰기:

mem[address] <= data; → 해당 주소에 데이터 저장

14. Parameter

상수 정의.

```
parameter SIZE = 8192;
```

장점:

- 재사용 가능
- 수정 용이

예:

```
parameter FIRMWARE = "mem.hex";
```

15. Localparam

내부 상수 정의.

```
localparam SIZE_WORD = 2'd2;
```

외부에서 변경 불가.

16. Initial Block

시뮬레이션 시작 시 1번 실행.

```
initial begin
```

```
end
```

예시:

```
initial begin
```

```
    $readmemh("mem.hex", mem);
```

```
end
```

메모리 초기화에 사용.

17. 시스템 함수

Verilog는 시뮬레이션용 시스템 함수를 제공한다.

예:

```
$display
```

```
$monitor
```

```
$readmemh
```

예시:

```
$display("value=%d", a);
```

18. Verilog 설계 방식

Verilog 설계는 크게 세 가지 방식으로 나뉜다.

Behavioral Modeling

동작 중심 설계

if

case

always

Dataflow Modeling

데이터 흐름 중심

assign

Structural Modeling

회로 연결 중심

module instantiation

Day1. LAB1 - Encoder

LAB 1에서는 Verilog HDL을 이용하여 4-to-2 Encoder를 설계하고, Testbench를 작성하여 simulation을 통해 동작을 검증하였다.

Encoder는 여러 입력 신호 중 활성화된 입력의 위치를 더 적은 비트 수의 binary code로 변환하는 조합논리회로(combinational logic circuit)이다.



din0	din1	din2	din3	dout[1]	dout[0]
1	0	0	0	0	0
0	1	0	0	0	1
0	0	1	0	1	0
0	0	0	1	1	1

이번 실습에서는 4개의 입력(din0 ~ din3) 중 하나만 1이라고 가정하고, 이를 2-bit 출

력(dout)으로 인코딩하였다.

입출력 관계는 다음과 같다.

- 0001 → 00
- 0010 → 01
- 0100 → 10
- 1000 → 11

정의되지 않은 입력(예: 여러 입력이 동시에 1인 경우)은 undefined case로 처리하였다.

Encoder는 조합논리회로이므로 always @* 구문이 사용되어있다.

always @*는 sensitivity list에 포함될 모든 입력 신호를 자동으로 추적하며, 입력 중 하나라도 변하면 내부 로직을 즉시 다시 계산한다. 따라서 클럭 없이 입력 변화에 즉각 반응하는 combinational logic 구현에 적합하다.

실습에서는 if-else 조건문을 사용하여 각 입력 조합에 대응하는 출력값을 정의하였다.

예를 들어 din0만 1이고 나머지 입력이 모두 0인 경우:

```
if (din0 == 1 && din1 == 0 && din2 == 0 && din3 == 0)
    dout = 2'b00;
```

Encoder는 기본적으로 한 번에 하나의 입력만 활성화되어야 한다. 그러나 실제 입력에서는 모든 입력이 0이거나, 둘 이상의 입력이 동시에 1이 되는 경우가 발생할 수 있다.

예를 들어 다음 입력들은 정상적인 Encoder 입력이 아니다.

- 0000
- 1111
- 0011

이러한 경우를 Undefined Case라고 한다.

본 실습에서는 샘플 코드의 default branch를 활용하여 Undefined Case를 처리하였다.

```
else begin
    dout = 2'bxx;
end
```

2'bxx는 Verilog에서 unknown value를 의미한다. 이는 simulation 과정에서 정상적인 binary output과 구분되며, 정의되지 않은 입력이 들어왔음을 쉽게 확인할 수 있게 한다.

또한 모든 경우에 대해 출력 값을 명시적으로 할당함으로써 불필요한 latch 생성도 방지할 수 있다는 점을 알게되었다.

시뮬레이션 결과는 아래 사진과 같다:



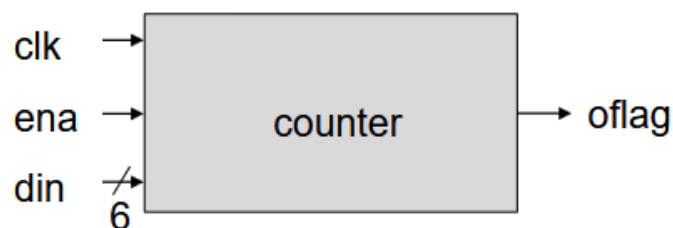
입력별 출력 결과는 다음과 같다:

- din0 활성 → dout = 00
- din1 활성 → dout = 01
- din2 활성 → dout = 10
- din3 활성 → dout = 11

이는 Encoder의 truth table과 일치하는 결과이다.

또한 Undefined Case 입력에서는 정상적인 binary output이 아닌 undefined 상태가 관찰되었다. 이를 통해 설계된 회로가 정상 입력과 비정상 입력을 구분하여 처리함을 확인할 수 있었다.

Day1. LAB2 – Counter



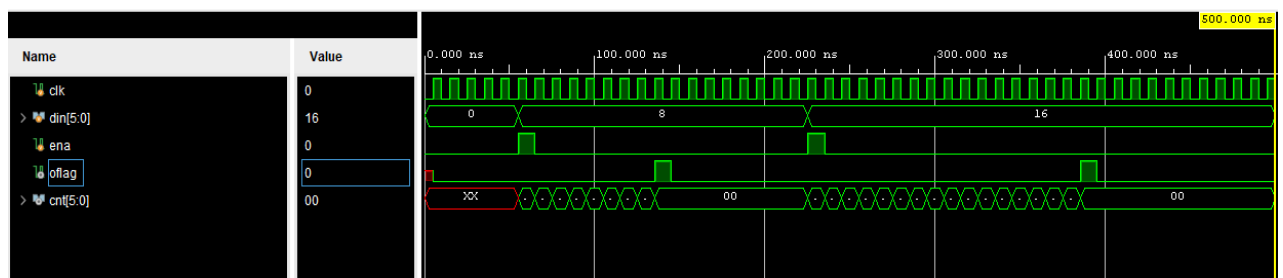
Counter 모듈은 내부 register인 cnt를 이용하여 현재 count 값을 저장한다. Counter는 clock의 positive edge마다 상태가 갱신되며, 따라서 combinational logic과 달리 입력이 변한다고 즉시 출력이 변하지 않고 clock edge에서만 값이 업데이트된다.

Counter 모듈의 주요 입출력은 다음과 같다.

- clk : Counter 동작의 기준이 되는 clock 신호
- din [5:0] : Counter에 load할 초기값
- ena : Counter 동작 시작 여부를 제어하는 enable 신호
- oflag : Counter 종료 직전 상태를 나타내는 flag 신호

din은 6비트 입력이므로 0부터 63까지 표현할 수 있다. 예를 들어 `din = 6'd8`은 counter 초기값을 8로 설정하는 것을 의미한다.

시뮬레이션 결과는 아래 사진과 같다:



waveform에서 `din = 6'd16`을 입력했는데 값이 10으로 보이는 경우가 있었다. 이는 값이 잘못된 것이 아니라 waveform의 radix가 hexadecimal(16진수)로 설정되어 있기 때문이다. Decimal 16은 hexadecimal로 0x10이므로 waveform에서는 10으로 표시된다.

ena는 counter가 새로운 값을 받아들일지 결정하는 제어 신호이다. ena=1일 때 din 값이 counter에 load되며, 이후 ena가 0으로 내려가면 counter는 countdown을 시작한다.

cnt는 실제 countdown 값을 저장하는 내부 register이다. 즉, 매 clock cycle마다 1씩 감소한다.

시뮬레이션에서 cnt의 값이 waveform에 보이면 더 직관적일 것이라고 생각하여 DUT 내부 signal을 직접 추가해서 같이 확인할 수 있도록 하였다.

oflag는 counter가 종료 시점에 도달했음을 나타내는 flag 신호이다. 실습 코드에서는 `cnt == 1`일 때 oflag가 1이 되도록 설계되었다.

겉보기에는 cnt가 0이 되었을 때 flag가 발생하는 것처럼 보일 수 있는데, 이는 cnt와 oflag가 동일한 clock edge에서 동시에 업데이트되기 때문이다(non-blocking

assignment). 즉, 내부적으로는 cnt가 1인 상태에서 oflag가 활성화되며, waveform에서는 다음 cycle의 cnt = 0과 함께 관찰될 수 있다.

- Test Case 1: Count from 8 to 0

첫 번째 실험에서는 din = 8을 입력하여 counter를 8로 초기화하였다. 이후 countdown이 시작되면서 counter 값은 다음과 같이 감소하였다.

8 7 6 5 4 3 2 1 0

정상적으로 1씩 감소하는 것을 확인할 수 있었다.

- Test Case 2: Count from 16 to 0

두 번째 실험에서는 din = 16을 입력하였다. Counter는 다음과 같이 감소하였다.

16 15 14 ... 3 2 1 0

Day1. LAB3 – Pulse Generator

Pulse Generator 모듈은 내부 변수인 state와 count를 이용하여 일정한 주기의 Pulse를 생성한다. state는 현재 출력이 High인지 Low인지를 저장하는 상태 변수이며, count는 Clock Cycle을 세는 Counter 역할을 수행한다. 출력 신호 W는 assign W = state를 통해 현재 state 값을 그대로 출력하도록 설계되어 있다.

Pulse Generator는 always @(negedge clk)에서 동작하므로 Clock의 negative edge마다 내부 상태와 Counter가 갱신된다. 따라서 조합회로(Combinational Logic)와 달리 입력이 변한다고 즉시 출력이 변하지 않고, Clock Edge에서만 출력이 변경되는 순차회로(Sequential Logic)의 특성을 가진다.

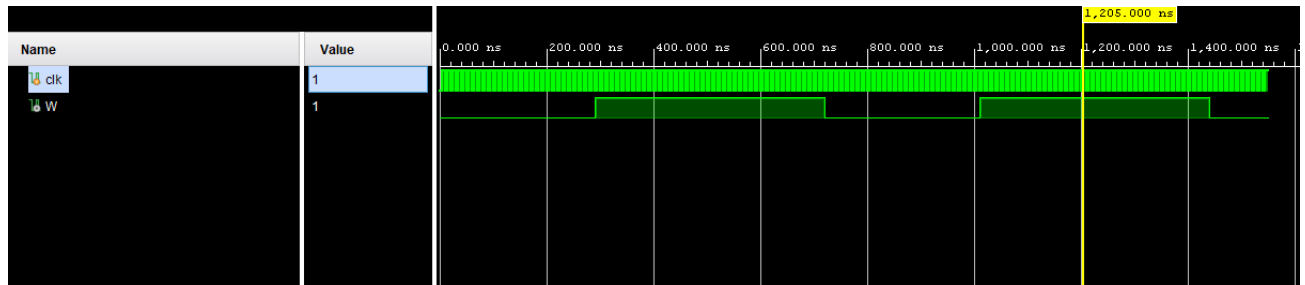
Pulse Generator 모듈의 주요 입출력 및 내부 변수는 다음과 같다.

- clk: Pulse Generator 동작의 기준이 되는 Clock 신호
- W: 생성되는 Pulse 출력 신호
- state: 현재 Pulse의 High 또는 Low 상태를 저장하는 내부 Register
- count: Clock Cycle을 저장하는 내부 Counter

초기 상태에서는 state = 0, count = 0으로 설정되므로 출력 W는 Low 상태를 유지한다. 이후 Clock의 negative edge마다 Counter가 증가하며 Low 상태에서는 Counter가 29에 도달하면 state가 1로 변경되어 출력 W가 High가 된다. 이때 Counter는 다시 1부터 증가를 시작한다. High 상태에서는 Counter가 43에 도달하면 state가 다시 0으로 변경되고 Counter가 초기화되어 다음 Pulse 생성 주기를 시작한다. 이러한 과정을 반복하여 29

Clock Cycle 동안 Low 상태를 유지한 후 43 Clock Cycle 동안 High 상태를 유지하는 Pulse가 지속적으로 생성되도록 구현하였다.

시뮬레이션 결과는 아래 사진과 같다.



Waveform을 보면 Clock의 negative edge마다 count가 증가하며 Pulse가 생성되는 것을 확인할 수 있다. 초기에는 state = 0이므로 출력 W는 Low 상태를 유지한다. count가 29에 도달하면 state가 1로 변경되어 출력 W가 High가 되며, count를 다시 1로 설정하여 High 구간을 카운트한다. 이후 count가 43에 도달하면 state가 다시 0으로 변경되고 count가 초기화되어 다음 Pulse 주기를 시작한다.

state는 현재 Pulse의 High와 Low 상태를 결정하는 내부 Register이며, count는 각 상태에서 경과한 Clock Cycle을 저장하는 Counter이다. 두 신호는 모두 Clock의 negative edge에서 갱신되므로 출력 W 역시 Clock Edge에서만 변경되는 순차회로의 특성을 가진다.

시뮬레이션 결과를 통해 출력 W가 29 Clock Cycle 동안 Low 상태를 유지한 후 43 Clock Cycle 동안 High 상태를 유지하는 Pulse를 반복적으로 생성하는 것을 확인하였다.

Pulse Generator를 시뮬레이션 하면서 순차회로의 동작 방식을 이해할 수 있었다. Counter와 State를 적절히 활용하면 원하는 주기의 Pulse를 생성할 수 있으며, 타이밍에 대한 제어 방법을 익힐 수 있었다.

Day2. LAB1 – Register file

RISC-V Register File은 CPU 내부에서 가장 자주 사용되는 데이터를 저장하는 고속 저장장치로, 32개의 32-bit Register로 구성되어 있다. 각 Register는 주소(Address)를 이용하여 접근하며, 하나의 Write Port와 두 개의 Read Port를 제공하여 하나의 Register에 데이터를 저장하는 동시에 두 개의 Register로부터 데이터를 읽을 수 있도록 설계되어 있다. Register File은 ALU를 비롯한 CPU의 여러 연산 장치와 직접 연결되어 명령어 수행 과정에서 필요한 데이터를 저장하고 전달하는 역할을 수행한다.

Register File은 Write 동작과 Read 동작의 특성이 서로 다르게 구현되어 있다. Write는

always @(posedge clk_i or negedge rstn_i) 블록에서 수행되므로 Clock의 Rising Edge에서만 데이터가 저장되는 순차회로(Sequential Logic)이며, Read는 always @(*) 블록을 이용하여 구현되어 Read Address가 변경되면 즉시 출력이 변경되는 조합회로(Combinational Logic)의 특성을 가진다.

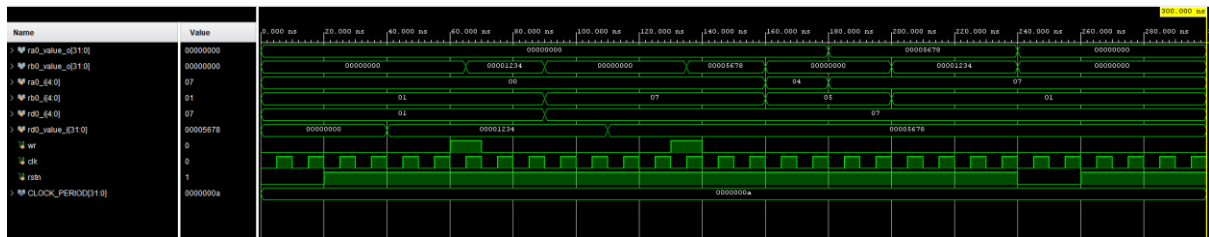
Register File 모듈의 주요 입출력 및 내부 변수는 다음과 같다.

- clk_i: Register File의 동작 기준이 되는 Clock 신호
- rstn_i: Active Low Reset 신호
- wr: Write Enable 신호
- rd0_i: 데이터를 저장할 Register의 주소
- rd0_value_i: Register에 저장할 32-bit 데이터
- ra0_i: 첫 번째 Read Address
- rb0_i: 두 번째 Read Address
- ra0_value_o: 첫 번째 Read Data 출력
- rb0_value_o: 두 번째 Read Data 출력
- reg_r1_q ~ reg_r31_q : 내부 Register를 저장하는 32-bit Register 변수
-

초기 상태에서는 rstn_i = 0으로 설정되어 모든 Register가 0으로 초기화된다. 이후 Reset이 해제되면 정상적인 동작을 시작한다. Write Enable(wr)이 1인 상태에서 Clock의 Rising Edge가 발생하면 rd0_i가 가리키는 Register에 rd0_value_i의 값이 저장된다. Testbench에서는 먼저 Register 1(R1)에 0x1234를 저장한 후, Register 7(R7)에 0x5678을 저장하도록 구성하였다. 저장이 완료된 이후에는 Read Address를 변경하여 원하는 Register의 값을 읽도록 하였다.

Read 동작은 Clock과 무관하게 수행된다. 먼저 R4와 R5를 읽도록 설정하였으며, 두 Register에는 값을 저장한 적이 없으므로 출력은 모두 0x00000000이 된다. 이후 Read Address를 R7과 R1로 변경하면 각각 0x00005678과 0x00001234가 출력되어 이전 Write 과정에서 저장한 데이터가 정상적으로 유지되고 있음을 확인할 수 있다. 마지막으로 Reset을 다시 수행하면 모든 Register가 다시 0으로 초기화되어 출력 역시 모두 0으로 변경되는 것을 확인하였다.

시뮬레이션 결과는 아래 사진과 같다:



첨부된 시뮬레이션 결과를 시간 순서에 따라 분석하면 다음과 같다.

rstn / wr / clk 구간

- rstn은 초기 구간(0~20ns) 동안 Low를 유지하다 20ns에 High로 전환되며, 이후 245ns 부근(정확히는 240~260ns 구간)에서 다시 짧게 Low로 떨어졌다가 High로 복귀한다.
- wr 신호는 60~70ns, 130~140ns 두 구간에서 짧게 High 펄스가 관찰되며, 이는 각각 R1과 R7에 대한 Write 동작과 정확히 일치한다.

rd0_i / rd0_value_i (Write 경로)

- rd0_i는 01 → 07로 한 번 전환되며, 이는 R1 Write 이후 R7을 Write 대상으로 지정한 것과 일치한다.
- rd0_value_i는 00000000 → 00001234 → 00005678 순으로 변화하며, 각 값이 유지되는 구간 중 wr 펄스가 걸린 시점(약 65ns, 135ns의 posedge)에 실제 레지스터 값이 반영된다.

ra0_i / ra0_value_o (Read Port A)

- ra0_i는 00(x0) → 04(x4) → 07(x7) 순으로 전환된다.
- ra0_i가 x7로 전환된 이후 짧게 00005678이 관찰되는데, 이는 앞서 R7에 Write된 값이 정확히 Read 포트로 출력됨을 보여준다.
- 이후 Reset(240~260ns) 구간을 지나면서 ra0_value_o는 다시 00000000으로 복귀하는데, ra0_i는 계속 x7을 가리키고 있음에도 값이 0으로 바뀐 것은 Reset이 R7의 저장값 자체를 지워버렸기 때문이다.

rb0_i / rb0_value_o (Read Port B)

- rb0_i는 01(x1) → 07(x7) → 05(x5) → 01(x1) 순으로 총 네 번 전환된다.
- rb0_i가 x1을 다시 가리키는 마지막 구간(200~240ns)에서는 00001234가 표시되며, 이는 R1에 Write된 값이 정확히 Read됨을 보여준다.
- 이후 Reset 구간을 지나면서 rb0_value_o 역시 00000000으로 복귀하는데, 이는 R1 저장값이 Reset에 의해 초기화되었기 때문이다.

Day2. LAB2 – RISC-V ALU

RISC-V ALU(Arithmetic Logic Unit)는 CPU 내부에서 산술 연산과 논리 연산을 수행하는 핵심 조합논리회로(Combinational Logic Circuit)이다. ALU는 두 개의 32-bit 피연산자(alu_a_i, alu_b_i)와 4-bit 연산 코드(alu_op_i)를 입력받아 지정된 연산을 수행한 후 32-bit 결과(alu_p_o)를 출력한다. 또한 연산 결과에 따른 상태 정보를 저장하기 위해 flcnz 플래그를 제공하며, Overflow, Carry, Negative, Zero 등의 예외 상황을 판단할 수 있도록 코드를 보완하였다.

ALU는 입력된 Opcode에 따라 하나의 연산만 수행하도록 case 문으로 구현되어 있으며, 지원하는 연산은 Arithmetic 연산(ADD, SUB), Logical 연산(AND, OR, XOR), Comparison 연산(SLT, SLTU), 그리고 Shift 연산(SLL, SRL, SRA)으로 구성된다. 모든 연산은 입력이 변경될 때마다 즉시 결과가 출력되는 always @(alu_op_i or alu_a_i or alu_b_i or sub_res_w) 블록에서 수행되므로 조합논리회로의 특성을 가진다.

ALU 모듈의 주요 입출력 및 내부 변수는 다음과 같다.

- alu_op_i: 수행할 연산을 선택하는 4-bit Opcode
- alu_a_i: 첫 번째 32-bit 피연산자
- alu_b_i: 두 번째 32-bit 피연산자
- alu_p_o: ALU 연산 결과 출력
- result_r: 연산 결과를 임시 저장하는 Register
- sub_res_w : SUB와 SLT 연산에서 사용하는 미리 계산된 뺄셈 결과
- shift_left*_r: Barrel Shifter의 Left Shift 중간 결과 저장
- shift_right*_r: Barrel Shifter의 Right Shift 중간 결과 저장
- shift_right_fill_r: Arithmetic Shift 시 Sign Bit를 채우기 위한 변수
- flcnz: 연산 결과 상태를 저장하는 Flag Register

또한 몇 가지 새롭게 알게 된 점을 정리해보았다. 우선 비교연산이다. 비교 연산은 Signed 비교(SLT)와 Unsigned 비교(SLTU)를 지원한다.

SLTU는 두 입력을 부호 없는 정수(Unsigned Integer)로 비교하며, alu_a_i가 alu_b_i보다 작으면 1을 출력하고 그렇지 않으면 0을 출력한다.

```
result_r = (alu_a_i < alu_b_i) ? 32'h1 : 32'h0;
```

SLT는 부호 있는 정수(Signed Integer)를 비교한다.

먼저 두 입력의 부호가 다른 경우에는 음수가 항상 더 작은 값이므로 alu_a_i의 최상위

비트(MSB)를 이용하여 결과를 결정한다.

```
if (alu_a_i[31] != alu_b_i[31])  
    result_r = alu_a_i[31] ? 32'h1 : 32'h0;
```

두 입력의 부호가 같은 경우에는 뺄셈 결과의 부호 비트를 이용하여 크기를 비교한다.

```
result_r = sub_res_w[31] ? 32'h1 : 32'h0;
```

이를 통해 부호가 다른 경우에도 Overflow의 영향을 받지 않고 정확한 Signed Comparison을 수행할 수 있다.

그 다음은 Shift 연산이다. Shift 연산은 SLL(Shift Left Logical), SRL(Shift Right Logical), SRA(Shift Right Arithmetic) 세 가지를 구현하였다.

Shift 연산은 일반적인 << 또는 >> 연산자를 사용하는 대신 Barrel Shifter 구조를 이용하여 구현하였다.

Shift 양은 최대 31bit이므로 Shift Amount의 하위 5bit(alu_b_i[4:0])만 사용하며,

- b[0] → 1bit Shift
- b[1] → 2bit Shift
- b[2] → 4bit Shift
- b[3] → 8bit Shift
- b[4] → 16bit Shift

를 순차적으로 수행하여 모든 Shift를 구현하였다.

예를 들어 Shift Amount가 13(01101₂)이라면

- 1bit Shift
- 4bit Shift
- 8bit Shift

가 차례대로 수행되어 총 13bit Shift가 이루어진다.

이러한 구조로 31개의 Shift 회로 대신 5단계의 Shift 회로만으로 모든 Shift를 수행할 수 있어 하드웨어 자원을 크게 절약할 수 있다는 것을 알게 되었다.

마지막으로 SRA이다. Arithmetic Shift Right(SRA)는 Signed Number를 위한 Shift 연산이다. SRL과 달리 오른쪽으로 이동하면서 비어 있는 상위 비트를 모두 0으로 채우는 것이 아니라 부호 비트(Sign Bit)를 그대로 복사한다.

이를 위해 코드에서는

shift_right_fill_r

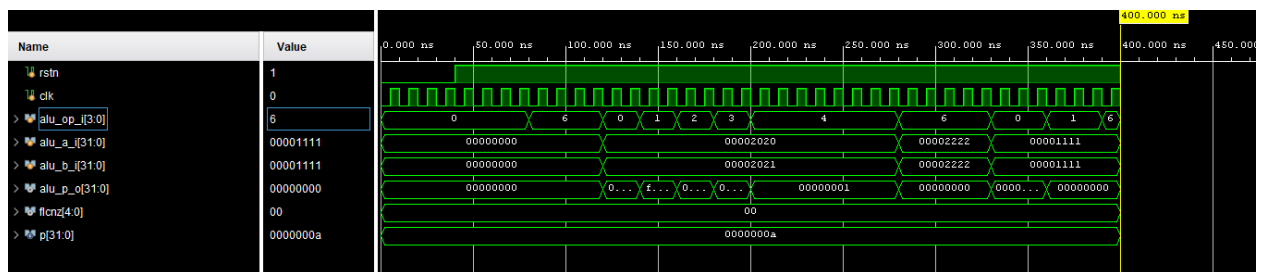
를 이용하여

- 양수 → 0000...
- 음수 → 1111...

를 미리 생성한 후 Shift 과정에서 상위 비트를 채우도록 구현하였다.

이를 통해 음수의 부호가 유지되는 Arithmetic Shift가 가능하다는 점을 알게 되었다.

시뮬레이션 결과는 아래와 같다:



시뮬레이션에서는 alu_op_i를 순차적으로 변경하여 ALU의 Arithmetic, Logical, Comparison 연산이 정상적으로 수행되는지 확인하였다. rstn은 High 상태를 유지하여 정상 동작을 수행하였으며, ALU는 조합논리회로이므로 Clock과 관계없이 입력이 변경될 때마다 출력(alu_p_o)이 즉시 변화하는 것을 확인할 수 있었다.

초기에는 alu_op_i = 6(SLTU)로 설정되어 alu_a_i와 alu_b_i가 모두 0x00000000인 상태에서 비교를 수행하였으며, 두 입력이 동일하므로 출력은 0x00000000이 되었다. 이후 약 100ns부터 입력이 0x00002020과 0x00002021로 변경되고 Opcode가 ADD, SUB, AND, OR, XOR 순으로 변경되면서 출력(alu_p_o) 역시 Opcode 변화에 따라 즉시 갱신되는 것을 확인하였고, 각 연산 결과가 정상적으로 출력되는 것을 확인하였다. 특히 SUB 연산에서는 0xFFFFFFFF가 출력되어 음수가 2의 보수 형태로 표현됨을 확인하였다.

약 300ns 이후에는 입력이 0x00002222로 변경되어 SLTU 비교를 수행하였으며, 두 값이 동일하므로 출력은 0이 되었다. 이후 입력을 0x00001111로 변경한 후에도 ADD, SUB 및 비교 연산이 정상적으로 수행되어 예상한 결과가 출력되는 것을 확인하였다.

전체 시뮬레이션 결과를 통해 Opcode의 변화에 따라 ALU가 산술 연산, 논리 연산 및 비교 연산을 정확하게 수행하였으며, 입력이 변경될 때마다 출력이 즉시 갱신되는 조합논리회로의 특성을 만족함을 확인하였다.

Day2. LAB3 – Multiplier

RISC-V Multiplier는 두 개의 32-bit 피연산자를 입력받아 64-bit 곱셈 결과를 생성하는 연산 모듈이다. 입력 데이터의 부호 여부를 판단하여 Signed와 Unsigned 곱셈을 모두 지원하며, 연산이 완료되면 mul_res_w를 통해 결과를 출력한다. Sequential Multiplier는 Clock을 이용하여 여러 사이클에 걸쳐 곱셈을 수행하므로 순차회로(Sequential Logic)의 특성을 가진다.

Multiplier 모듈의 주요 입출력 및 내부 변수는 다음과 같다.

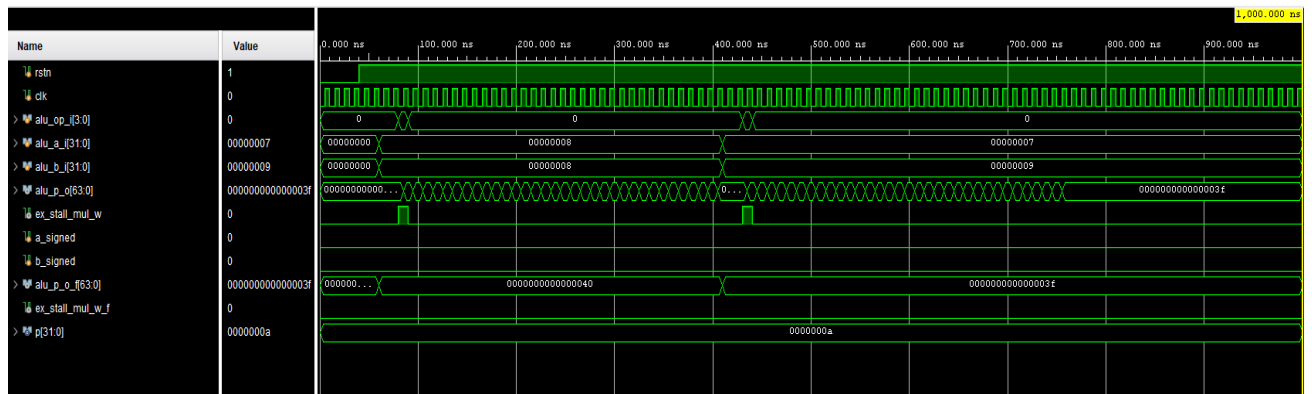
- clk_i : Multiplier 동작을 위한 Clock
- rstn_i : Active Low Reset
- id_ra_value_r : 첫 번째 32-bit 피연산자(Multiplicand)
- id_rb_value_r : 두 번째 32-bit 피연산자(Multiplier)
- id_a_signed_r, id_b_signed_r : Signed/Unsigned 여부를 결정하는 신호
- mul_res_w : 64-bit 곱셈 결과
- ex_stall_mul_w : 곱셈 수행 중 Pipeline Stall 신호
- mul_busy_r : 곱셈 수행 여부
- mul_ready_r : 곱셈 완료 여부
- mul_count_r : 곱셈 반복 횟수를 저장하는 Counter
- mul_res_r : 곱셈 중간 결과 저장 Register
-

Sequential Multiplier는 Shift-and-Add 알고리즘을 이용하여 곱셈을 수행한다.

Multiplier의 LSB가 1이면 Multiplicand를 상위 비트에 더하고, 이후 결과 전체를 1bit 오른쪽으로 Shift한다. 이러한 과정을 Counter가 종료될 때까지 반복하여 최종 곱셈 결과를 생성한다. 또한 입력이 Signed Number인 경우에는 먼저 음수 여부를 판단하여 절댓값으로 변환한 후 곱셈을 수행하고, 연산이 종료된 후 두 입력의 부호가 서로 다른 경우에만 결과를 다시 음수로 변환한다.

Counter(mul_count_r)는 32에서 시작하여 매 Clock마다 1씩 감소하며, Counter가 0이 되면 곱셈이 종료되고 mul_busy_r가 해제되어 결과가 출력된다. 반면 Fast Multiplier는 Shift-and-Add를 수행하지 않고 Verilog의 전용 곱셈기(*)를 사용하여 한 번에 64-bit 결과를 생성한다. 따라서 별도의 Counter나 Stall 과정 없이 즉시 결과가 출력되며, ex_stall_mul_w도 항상 0으로 유지된다.

시뮬레이션 결과는 다음 사진과 같다:



시뮬레이션에서는 alu_op_i를 변경하여 Multiplier의 동작을 확인하였다. 초기에는 Reset이 해제된 상태(rstn=1)에서 입력값이 모두 0으로 설정되어 있으며, 곱셈 결과(alu_p_o_r[63:0])와 Fast Multiplier의 출력(ex_sum_mul_w_f)도 모두 0으로 나타난다.

약 50ns에서 첫 번째 테스트가 시작된다. 입력값은 alu_a_i = 8, alu_b_i = 8로 설정되며 Sequential Multiplier가 동작하기 시작한다. 이때 ex_stall_mul_w가 High로 변하여 곱셈 연산이 진행 중임을 나타내며, 내부에서는 Shift-and-Add 알고리즘에 따라 여러 Clock Cycle 동안 연산이 수행된다. 연산이 완료된 후 ex_stall_mul_w가 Low로 변경되고 최종 결과인 0x0000000000000040(64)이 출력되는 것을 확인할 수 있다. Fast Multiplier(ex_sum_mul_w_f) 역시 동일한 결과를 즉시 출력하여 두 구현 방식의 결과가 일치함을 확인하였다.

약 350ns에서는 두 번째 테스트가 수행된다. 입력값이 alu_a_i = 7, alu_b_i = 9로 변경되며 다시 ex_stall_mul_w가 High가 되어 Sequential Multiplier가 연산을 수행한다. 여러 Clock Cycle 이후 Stall 신호가 해제되면서 최종 결과 0x000000000000003F(63)이 출력되었으며, Fast Multiplier 역시 동일한 결과를 출력하였다.

시뮬레이션 결과를 통해 Sequential Multiplier는 곱셈이 수행되는 동안 ex_stall_mul_w를 활성화하여 여러 Clock Cycle에 걸쳐 연산을 수행하는 반면, Fast Multiplier는 별도의 Stall 없이 동일한 곱셈 결과를 즉시 생성함을 확인하였다. 또한 두 테스트 모두에서 두 Multiplier의 출력이 동일하게 나타나 설계한 곱셈 회로가 정상적으로 동작함을 검증하였다.

Day3. LAB1 – Memory Model

Memory module은 프로세서가 실행할 instruction과 프로그램 실행 중 필요한 data를 저장하는 역할을 한다. CPU는 Program Counter가 가리키는 주소를 기반으로 instruction memory에서 명령어를 fetch하고, 연산 과정에서 필요한 값들은 data memory를 통해 읽거나 저장한다.

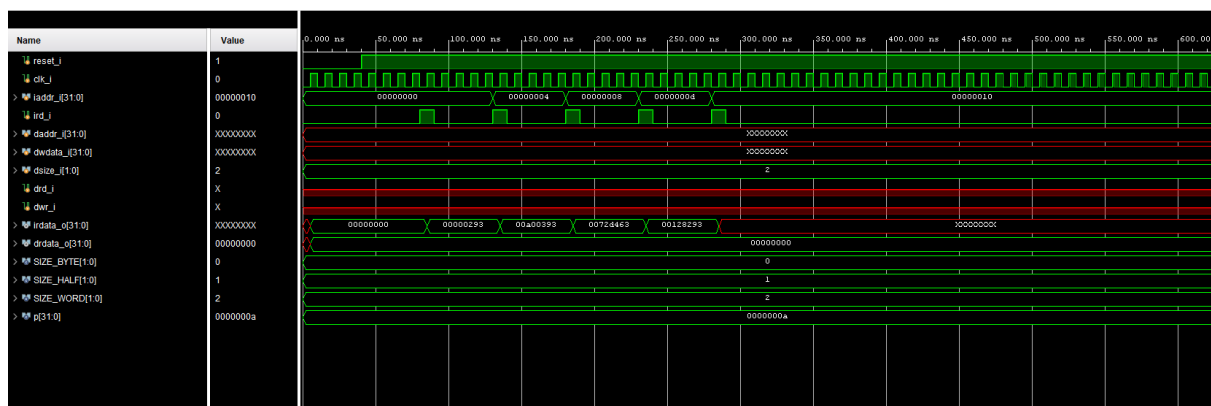
Memory module의 주요 신호는 다음과 같다.

- clk_i : Memory 동작의 기준이 되는 clock signal
- reset_i : 내부 register 초기화 신호
- iaddr_i [31:0] : Instruction memory address
- ird_i : Instruction read enable signal
- irdata_o [31:0] : Read된 instruction output
- daddr_i / ddata_i / drdata_o : Data memory read/write 관련 신호

Memory는 32-bit word 단위 array로 구성되며, simulation 시작 시 mem.hex 파일의 machine code가 memory에 로드된다. Instruction read는 combinational 방식이 아닌 synchronous read 방식으로 구현되어, read enable signal이 활성화되더라도 즉시 출력되지 않고 clock의 positive edge에서 값이 갱신된다.

Instruction address는 byte address 형태이므로 실제 memory indexing 시 하위 2비트를 제외한 주소를 사용한다. 이를 통해 4-byte 단위 instruction fetch가 가능하다.

진행한 시뮬레이션 결과는 다음 사진과 같다:



Waveform에서 보이듯, reset_i가 활성화된 이후 ird_i를 주기적으로 1로 설정하여 여러 instruction address에 대해 read operation을 수행하였다. 그 결과 irdata_o에서 memory

에 저장된 instruction들이 순차적으로 정상 출력되는 것을 확인할 수 있었다.

또한, ird_i가 활성화된 직후 즉시 값이 바뀌는 것이 아니라, 다음 positive edge 이후 irdata_o가 업데이트되는 것을 확인할 수 있다. 이는 설계한 memory가 synchronous read 방식으로 동작함을 보여준다.

그리고, instruction address가 증가함에 따라 memory 내부의 서로 다른 instruction들이 순차적으로 fetch되는 것을 확인할 수 있었다. 각 address에 대해 예상한 machine code가 정확히 출력되었으므로 memory indexing이 정상적으로 수행됨을 알 수 있다.

Simulation 초반에는 output signal이 X 또는 0으로 보이는 구간이 존재하였다. 이는 reset 이전이거나 instruction read가 아직 수행되지 않은 시점이기 때문이며, reset 이후 read signal이 활성화되면서 정상적인 instruction 값이 출력되었다.

실습 과정에서는 instruction output이 계속 0으로 출력되는 문제가 발생하기도 했다. 초기에는 read logic 문제를 의심하였으나 로그를 찍어보니 mem.hex 파일이 시뮬레이션 환경에서 정상적으로 load되지 않았기 때문이었다. 이후 hex 파일 경로를 올바르게 설정하여 문제를 해결할 수 있었다.

이번 실습을 통해 RISC-V memory module의 기본 구조와 instruction fetch 과정을 이해할 수 있었다. 특히 instruction read가 clock edge 기반의 synchronous 방식으로 동작한다는 점과, byte address를 word address로 변환하여 memory indexing을 수행한다는 점을 확인할 수 있었다.

Day3. LAB2 – Decoder

Decoder는 CPU pipeline에서 Instruction Decode (ID) stage에 해당하며, instruction의 binary code를 분석하여 해당 명령어가 어떤 연산을 수행해야 하는지 판단하는 역할을 한다. 즉, Fetch 단계에서 메모리로부터 읽어온 32-bit instruction을 입력받아 다음 단계에서 필요한 정보들을 추출한다.

주요 출력 정보는 다음과 같다.

- Immediate value (id_imm_w)
- Destination register index (id_rd_index_w)
- Source register index (id_ra_index_w, id_rb_index_w)
- ALU operation code (id_alu_op_w)
- Branch operation (id_branch_w)
- Memory access size (id_mem_size_w)

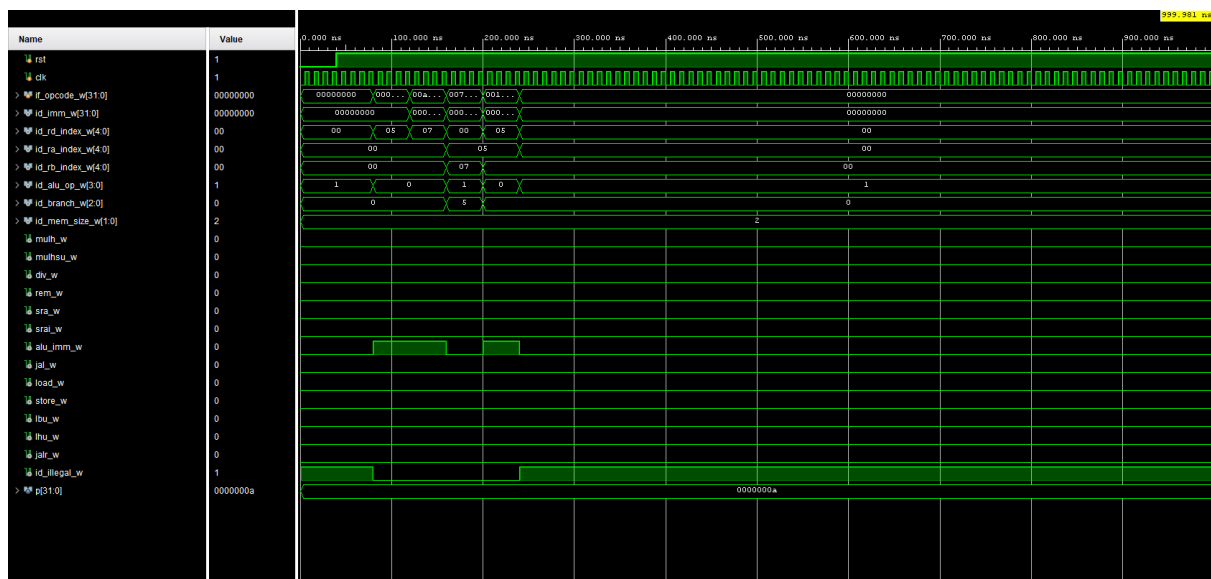
- 각종 control flag (load, store, jump, illegal 등)

Decoder는 조합논리(combinational logic) 기반으로 설계되었기 때문에 clock edge를 기다리지 않고, 입력 instruction이 변화하는 즉시 출력값이 갱신된다. 따라서 instruction decoding 결과를 빠르게 다음 stage로 전달할 수 있다.

RISC-V instruction은 기본적으로 32-bit 길이를 가지며, opcode, rd, funct3, rs1, rs2, funct7과 같은 여러 field로 구성된다. 이 중 opcode는 instruction의 큰 종류를 결정하며, funct3와 funct7은 같은 opcode 내부에서 보다 세부적인 연산 종류를 구분하는 역할을 한다. 예를 들어 opcode가 0010011이면 ALU Immediate instruction, 0110011이면 ALU Register instruction, 1100011이면 Branch instruction으로 해석된다.

이후 Decoder는 funct3 및 funct7 값을 추가로 분석하여 실제 명령어를 구분한다. 예를 들어 같은 ALU 계열 instruction이라도 addi, srai, lw, bge 등은 funct field 조합에 따라 서로 다른 연산으로 decode된다. 이를 통해 Decoder는 하나의 32-bit binary instruction으로부터 CPU가 수행해야 할 정확한 동작을 결정할 수 있다.

시뮬레이션 결과는 아래 사진과 같다:



시뮬레이션에서는 4개의 RISC-V instruction을 순차적으로 Decoder에 입력하고, 각 instruction에 대해 출력되는 control signal의 변화를 관찰하였다. 주요 확인 대상은 immediate value, register index, ALU operation, branch operation, 그리고 illegal flag였다.

초기 reset 구간에서는 유효한 instruction이 입력되지 않았으므로 대부분의 출력 signal은 기본값을 유지하며, id_illegal_w가 1로 활성화되어 invalid instruction 상태를 나타냈다.

첫번째 instruction 0x00000293은 addi x5, x0, 0에 해당한다. Simulation 결과

destination register는 x5, immediate value는 0으로 decode되었으며 ALU operation은 ADD로 설정되었다.

두번째 instruction 0x00a00393은 addi x7, x0, 10에 해당하며, destination register는 x7, immediate value는 10으로 정상 decode되었다.

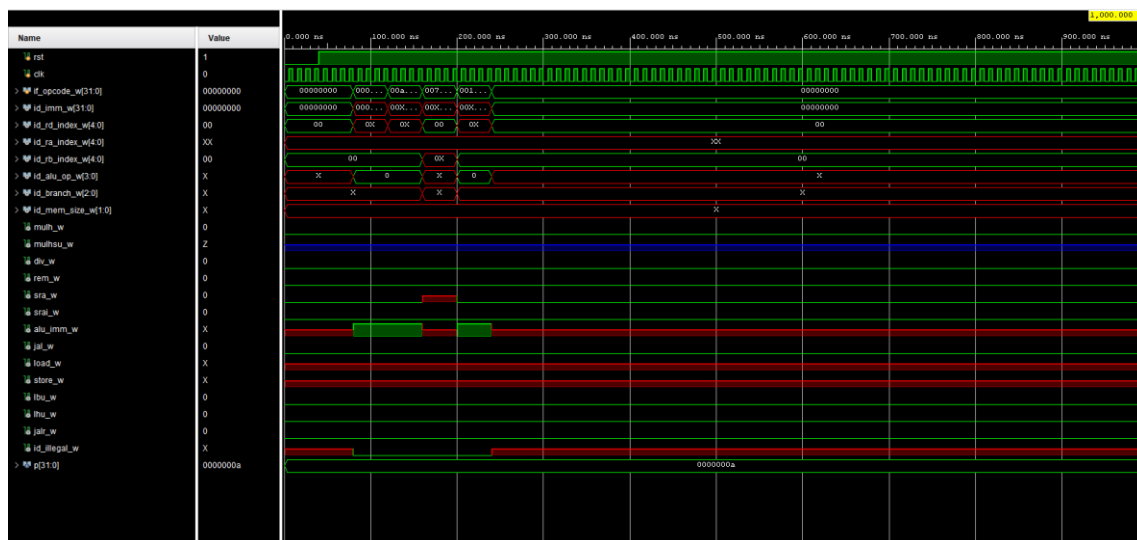
세번째 instruction 0x0072d463은 branch instruction인 bge x5, x7, offset이다. 이때 source register가 각각 x5와 x7로 decode되었고, id_branch_w 값이 5로 출력되어 BR_GE 조건이 정상적으로 인식됨을 확인하였다.

네번째 instruction 0x00128293은 addi x5, x5, 1로, source register와 destination register 모두 x5이며 immediate value는 1로 decode되었다.

앞서 언급했듯 Waveform의 형태를 통해 instruction이 바뀔 때마다 Decoder 출력이 즉시 변화하는 것을 확인할 수 있었다.

실습 과정에서 초기 simulation 결과에서는 일부 signal이 X 상태로 출력되는 문제가 발생하였다. branch 관련 signal과 일부 control signal이 정상적으로 출력되지 않았다.

아래는 당시 문제가 발생한 사진이다.



문제 원인을 분석하는 과정에서 Decoder 내부 wire 선언과 testbench 연결 상태를 다시 점검하였다. 초기에는 branch instruction decode가 실패한 것으로 보였으나, 내부 signal 선언 누락 및 simulation 설정 문제를 발견하였다.

또한 branch instruction의 opcode와 funct3 값을 직접 확인하여 Decoder가 올바른 path를 통과하는지 검증하였다. 수정 후에는 모든 instruction이 정상적으로 decode되었으며, branch signal 역시 의도한 값으로 출력되었다.

이러한 과정을 통해 Decoder는 CPU가 instruction을 이해하고 적절한 control signal을 생성하기 위한 핵심 모듈이며, opcode parsing과 immediate generation이 핵심 기능임을 배울 수 있었다.

Day3. LAB3 – Memory + Decoder

이번 실습에서는 이전 Lab에서 구현한 Memory와 Decoder를 하나의 시스템으로 연결하여 Instruction Fetch와 Decode 과정이 정상적으로 동작하는지 확인하였다. Memory는 mem.hex에 저장된 RISC-V machine code를 읽어오는 역할을 수행하며, Decoder는 Memory에서 전달받은 32-bit instruction을 해석하여 CPU에서 사용할 control signal을 생성한다. 즉, Memory는 Instruction Fetch(IF) stage의 역할을 담당하고, Decoder는 Instruction Decode(ID) stage의 역할을 담당한다.

Memory는 instruction address(iaddr_i)를 입력받아 해당 주소의 instruction을 읽어 irdata_o로 출력한다. Testbench에서는 instruction address를 4-byte 단위(0x0, 0x4, 0x8, 0xC, 0x10)로 증가시키면서 순차적으로 instruction을 읽도록 구성하였다. Memory에서 출력된 instruction은 Decoder의 입력(if_opcode_w)으로 전달되어 opcode, register index, immediate value, ALU operation 등을 분석한다.

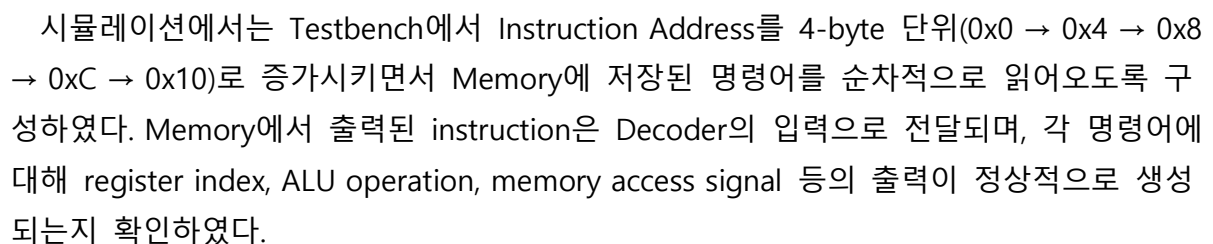
이번 실습에서는 아래와 같은 RISC-V 프로그램을 mem.hex 파일에 machine code 형태로 저장하여 사용하였다.

```
add x9, x8, x0
add x10, x0, x0
add x11, x0, x0
Loop:
lw x12, 0(x9)
add x10, x10, x12
addi x9, x9, 4
addi x11, x11, 1
addi x13, x0, 64
blt x11, x13, Loop
```

위 Assembly를 RV32I machine code로 변환하여 mem.hex에 저장하였다.

```
000404b3
00000533
```

시뮬레이션 결과는 아래 사진과 같다:



두 번째 명령어 0x00000533은 add x10, x0, x0으로 decode되었으며, 목적지 레지스터는 x10, 소스 레지스터는 모두 x0로 올바르게 출력되었다.

세 번째 명령어 0x000005b3은 add x11, x0, x0으로 해석되었으며, 목적지 레지스터가 x11로 정상적으로 decode되는 것을 확인하였다.

네 번째 명령어 0x0004a603은 lw x12, 0(x9)에 해당한다. Destination register는 x12, Source register는 x9로 decode되었으며, Load instruction으로 인식되어 Memory Access 관련 Control Signal이 활성화되는 것을 확인하였다.

다섯 번째 명령어 0x00c50533은 add x10, x10, x12에 해당하며, 목적지 레지스터는 x10, 소스 레지스터는 x10과 x12로 정상적으로 decode되었다. 이를 통해 Load 명령으로 읽어온 데이터를 이후 ALU 연산에서 사용할 수 있도록 필요한 정보가 올바르게 추출되는 것을 확인하였다.

전체 Waveform을 확인한 결과 Instruction Address가 증가할 때마다 Memory에서 해당 Machine Code를 정상적으로 읽어왔으며, Decoder의 출력 또한 입력 Instruction의 변화에 따라 즉시 갱신되는 것을 확인하였다. 이를 통해 Memory와 Decoder가 정상적으로 연결되어 RISC-V Processor의 Instruction Fetch와 Instruction Decode 과정이 올바르게 수행됨을 확인하였다.

이를 통해 RISC-V 명령어가 Memory에서 읽혀 Decoder에서 해석되는 과정을 이해할 수 있었으며, Memory와 Decoder를 연동하여 CPU의 초기 동작 과정을 확인할 수 있었다.